AI-Based Student Support Assistant

Design and Implementation of a Conversational Agent using Google Dialogflow

1. Introduction and Problem Statement

A university wants to develop an AI-based Student Support Assistant that provides instant responses to student queries related to course registration, timetable, examination schedules, attendance details, and campus services. As an AI developer, the task is to design and implement a conversational agent using Dialogflow that can understand user requests, process intents, and provide appropriate responses.

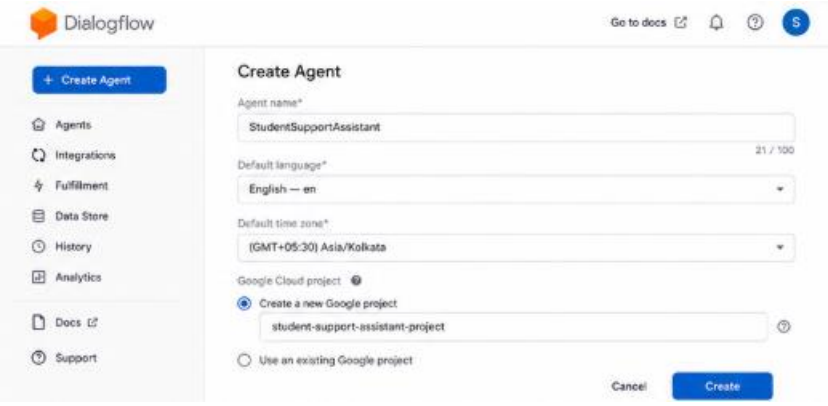
The core problems this assistant addresses are:

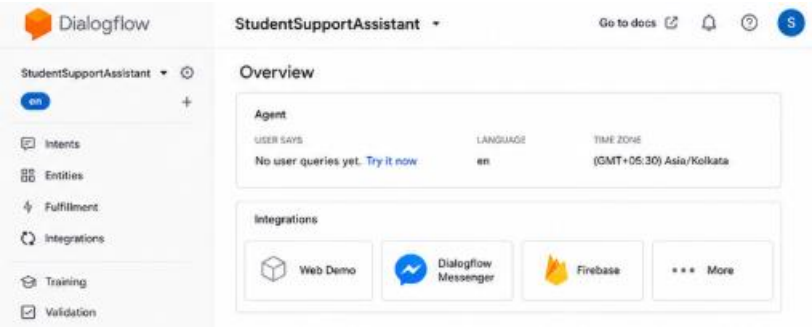
- Students repeatedly ask the same set of questions (timetable, exam dates, attendance %, registration deadlines) through email, phone, or in person.
- Administrative staff spend disproportionate time on queries that could be automated.
- Students expect instant, 24/7 responses, which human staff cannot always provide.

The proposed solution is a Dialogflow-based chatbot that classifies incoming text into predefined intents (e.g., "Check Attendance", "Get Exam Schedule"), extracts key parameters (e.g., course name, semester), and either responds directly or calls a backend webhook to fetch live data from the university's database.

4. Step 1 — Creating a Dialogflow Agent

1. Go to https://dialogflow.cloud.google.com/ and sign in with your Google account.
2. Click "Create Agent" in the left-hand sidebar.
3. Enter the agent name: StudentSupportAssistant.
4. Set the default language to English – en, and default time zone to your local time zone.
5. Choose "Create a new Google project" (or select an existing GCP project) and click Create.
6. Wait for Dialogflow to provision the agent — this takes a few seconds and creates the underlying GCP project automatically.





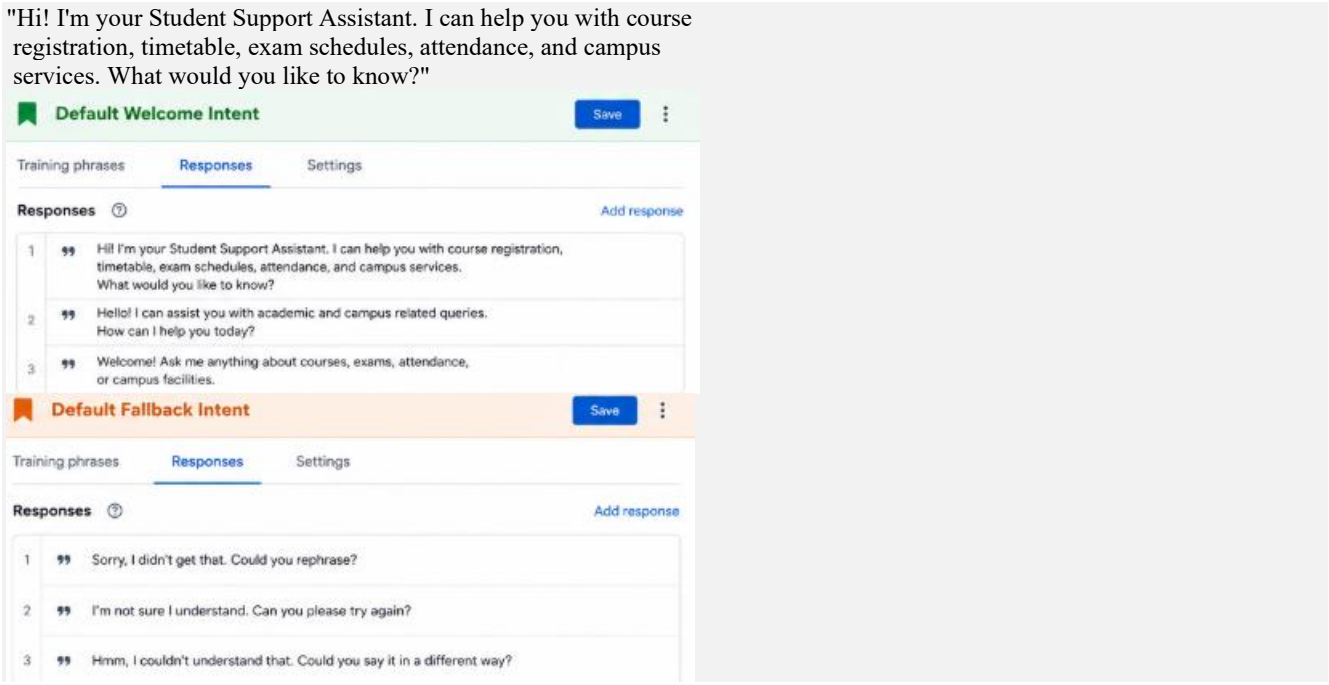
5. Step 2 — Understanding the Default Intents

Every new agent is created with two default intents:

- Default Welcome Intent — triggered when the conversation starts; used to greet the student and explain what the assistant can help with.
- Default Fallback Intent — triggered when Dialogflow cannot match the user's input to any intent; used to politely ask the student to rephrase.

Customize the Default Welcome Intent's responses, e.g.:

"Hi! I'm your Student Support Assistant. I can help you with course registration, timetable, exam schedules, attendance, and campus services. What would you like to know?"



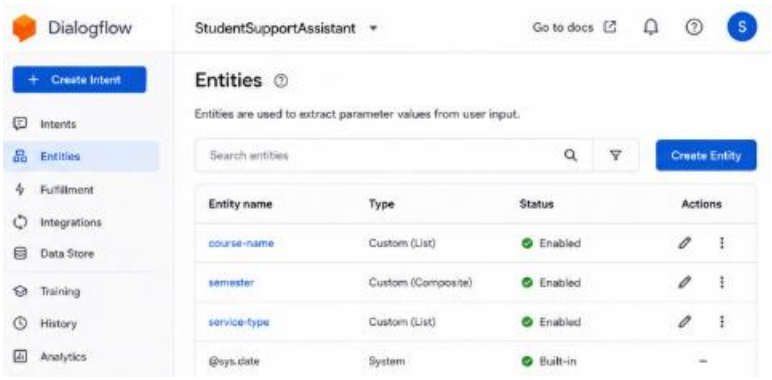
6. Step 3 — Designing Custom Entities

Entities let Dialogflow extract structured data (parameters) from free text. For this assistant, the following custom entities are created under the Entities section:

Entity Name	Type	Sample Values / Synonyms
@course-name	Custom (List)	Data Structures, DBMS, Operating Systems, Computer Networks

@semester	Custom (Composite / Number)	1st, 2nd, 3rd ... 8th semester
@service-type	Custom (List)	hostel, library, bus pass, ID card, scholarship
@date (system)	@sys.date	Built-in Dialogflow system entity — no setup needed

7. Click Entities in the left sidebar, then "Create Entity".
8. Name the entity (e.g., course-name).
9. Add reference values and synonyms row by row (e.g., reference value 'DBMS' with synonyms 'Database Management Systems', 'Databases').
10. Enable "Allow automated expansion" if you want Dialogflow to recognize similar unseen values.
11. Click Save. Repeat for semester and service-type.



7. Step 4 — Creating Intents for Student Queries

Each type of student query is modeled as a separate intent. For every intent: add 10–20 varied training phrases, annotate parameters by highlighting words in the phrase and mapping them to entities, and write one or more static or dynamic responses.

7.1 Intent: check.attendance

Training phrases:

- What is my attendance in @course-name ?
- Show my attendance percentage for @course-name
- How many classes have I attended in @course-name this @semester ?

Parameters: course-name (@course-name, required), semester (@semester, optional).

Sample response: "Your attendance in {course-name} is 82%. You need at least 75% to be eligible for exams." (Note: for a production assistant this value would come from a webhook, not a static response — see Step 7.)

7.2 Intent: get.exam.schedule

Training phrases:

- When is my @course-name exam?
- Tell me the exam timetable for @semester semester
- What date is the Operating Systems exam?

Parameters: course-name (@course-name, optional), semester (@semester, optional).

7.3 Intent: course.registration

Training phrases:

- How do I register for @course-name ?
- I want to enroll in @course-name for @semester semester
- What is the last date for course registration?

7.4 Intent: get.timetable

Training phrases:

- Show me my class timetable
- What classes do I have today?
- Send me the @semester semester timetable

7.5 Intent: campus.services

Training phrases:

- How do I apply for a @service-type ?
- I need information about the @service-type
- Where is the @service-type office located?

8. Step 5 — Using Contexts for Multi-Turn Conversations

Contexts let the assistant remember information across turns. For example, after a student asks about a course, a follow-up question like "What about the exam?" should still know which course was meant.

12. Open the check.attendance intent, scroll to Contexts, and add an output context named course-context with a lifespan of 5 turns.
13. Create a follow-up intent (e.g., get.exam.schedule.followup) that sets course-context as an input context, so it only triggers when a course was mentioned recently.
14. In the follow-up intent, reuse the #course-context.course-name parameter instead of asking the student again.

9. Step 6 — Slot Filling for Required Information

When a required parameter is missing from the student's query, Dialogflow automatically asks a follow-up question (a "prompt") until it is provided. For example, if a student types "What is my attendance?" without naming a course, Dialogflow should ask "Which course would you like to check attendance for?"

15. In the intent's Action and Parameters table, mark the course-name parameter as Required.
16. Click the speech-bubble icon next to the parameter to open the Prompts dialog.
17. Add prompt text: "Sure — which course would you like to check?"
18. Save the intent and test with an incomplete query in the simulator.

10. Step 7 — Enabling Fulfillment (Webhook)

Static responses work for demonstration, but a production assistant should fetch live data (actual attendance %, actual exam dates) from the university's database. This is done via a webhook.

19. Go to Fulfillment in the left sidebar and enable the Webhook toggle.
20. Enter the webhook URL of your backend service, e.g., a Google Cloud Function endpoint: https://REGION-PROJECT.cloudfunctions.net/studentAssistantWebhook.
21. Save. In each intent that should call the backend (e.g., check.attendance), scroll to Fulfillment at the bottom of the intent page and enable "Enable webhook call for this intent".
22. On the backend, write a Node.js/Python handler that reads the intent name and parameters from the request, queries the database, and returns a JSON response in Dialogflow's expected format.

Example minimal webhook handler (Node.js, using the dialogflow-fulfillment library):

```
const functions = require('firebase-functions');
const { WebhookClient } = require('dialogflow-fulfillment');

exports.studentAssistantWebhook = functions.https.onRequest((req, res) => {
  const agent = new WebhookClient({ request: req, response: res });

  function checkAttendance(agent) {
    const course = agent.parameters['course-name'];
    const attendance = getAttendanceFromDB(course); // your DB lookup
    agent.add(`Your attendance in ${course} is ${attendance}%.`);
  }

  let intentMap = new Map();
  intentMap.set('check.attendance', checkAttendance);
  agent.handleRequest(intentMap);
});
```

11. Step 8 — Testing in the Dialogflow Simulator

The right-hand panel of the Dialogflow console is a built-in test simulator. Type sample queries and confirm the correct intent, parameters, and response are returned.

Suggested test cases:

- "What is my attendance in DBMS?" → should trigger check.attendance with course-name = DBMS.
- "When is my exam?" → should trigger get.exam.schedule and, if course-name is required, prompt for it.
- "How do I apply for a hostel?" → should trigger campus.services with service-type = hostel.
- "asdkjhasd" (gibberish) → should trigger the Default Fallback Intent.

23. For each test case, use the "Diagnostic Info" panel (the </> icon) to inspect the matched intent, confidence score, and extracted parameters in raw JSON.

14. Sample Conversation Transcripts

Student: Hi

Assistant: Hi! I'm your Student Support Assistant. I can help you with course registration, timetable, exam schedules, attendance, and campus services. What would you like to know?

Student: What is my attendance in DBMS?

Assistant: Your attendance in DBMS is 82%. You need at least 75% to be eligible for exams.

Student: What about my exam date?

Assistant: Your DBMS exam is scheduled for [date from database].

Student: How do I apply for a hostel?

Assistant: You can apply for a hostel through the Student Portal → Hostel Services tab. The office is located in Block C, Room 12.

- ontext too soon or hold onto stale context too long.